

Part 2: UML Diagram Creation

The following section contains two UML diagrams created for the CampusPass – Student Entry & Attendance Tracking System:

Use Case Diagram – Shows the interactions between actors (Student, Security Guard, Admin) and the system.

Activity Diagram – Illustrates the step-by-step process flow of student entry, attendance recording, and report generation.

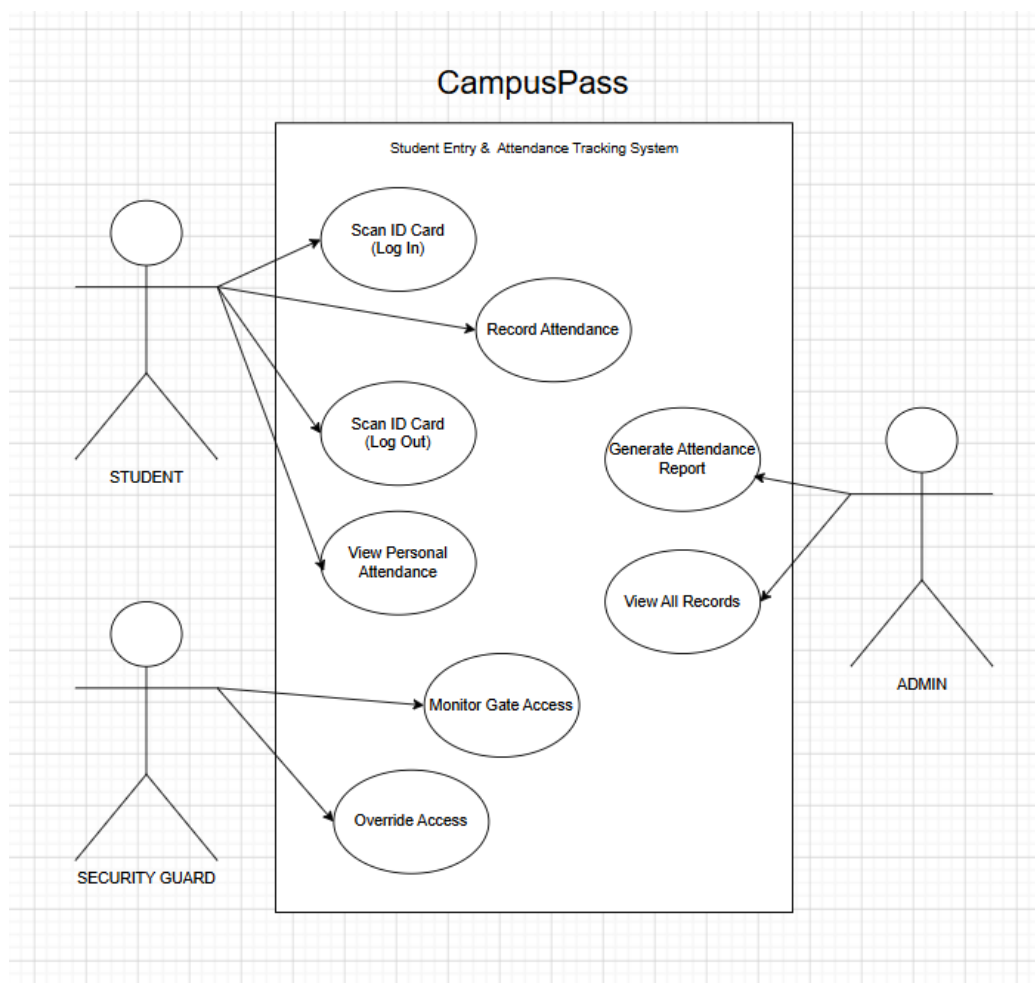


Figure 1. Use Case Diagram

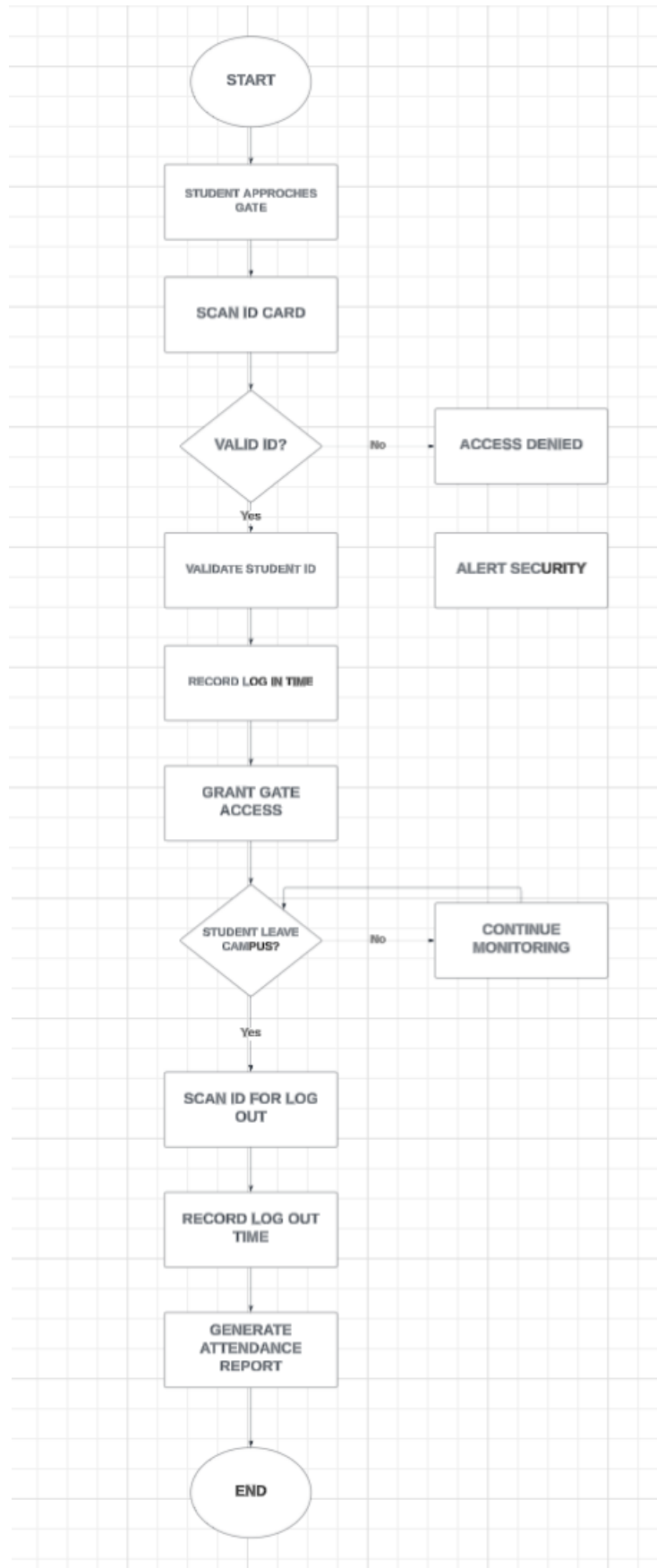


Figure 2. Activity Diagram

Part 3: SDLC Application

The development of the **CampusPass – Student Entry & Attendance Tracking System** followed the Software Development Life Cycle (SDLC) to ensure a structured and reliable approach.

Planning Phase: During planning, requirements were gathered through interviews and consultations with campus security personnel, faculty members, and students. These discussions identified the need for a secure ID scanning mechanism at entry and exit points, standardized attendance report formats for faculty, and clear access control rules for security staff. This phase established the scope, objectives, and constraints of the system, ensuring alignment with institutional policies and user expectations.

Design Phase: The design stage involved creating UML diagrams, including Use Case, Activity, and Class diagrams to visualize system interactions, workflows, and data structures. A database schema was also designed to store student profiles, attendance records, and timestamps, emphasizing normalization and data integrity. The design ensured that both functional requirements (such as login/logout and attendance tracking) and non-functional requirements (such as security and scalability) were addressed.

Implementation Phase: In implementation, the system was developed with integration to ID scanners at campus gates, a web-based dashboard for administrators to generate and view reports, and a mobile application for students to access their personal attendance records. Programming languages such as Java and Python were used for back-end and application logic, while MySQL served as the database management system. This phase translated the design into a working prototype and eventually into a deployable system.

Testing Phase: Testing was conducted at multiple levels to ensure system reliability. Unit testing validated the core functions such as ID scanning and timestamp recording. Integration testing confirmed seamless connectivity between the application and the database. Finally, user acceptance testing was carried out with actual students, faculty, and security staff to evaluate usability, accuracy, and overall satisfaction. Feedback from this phase was used to refine the system before deployment.

Part 4: Reflection

1. What did you learn from creating UML diagrams? Through the process of creating UML diagrams, I realized how essential visual modeling is in system development. Use Case diagrams provided clarity on the different actors involved such as students, security guards, and administrators and the specific functions they require from the system. This helped me understand not only *who* uses the system but also *why* each feature exists. Meanwhile, Activity diagrams offered a step-by-step representation of workflows, such as scanning IDs, validating entries, and recording attendance. By mapping these processes visually, I was able to identify potential errors, redundancies, and missing steps early on, which made the development process more efficient and reduced the risk of miscommunication among team members. Overall, UML diagrams transformed abstract requirements into concrete, structured models that guided both design and implementation.

2. Why is planning (SDLC) important before building a system? Applying the Software Development Life Cycle (SDLC) taught me the importance of structured planning before writing any code. The planning phase ensured that all requirements were gathered systematically, preventing costly mistakes that often arise from unclear objectives. It also provided a roadmap that guided the team through design, implementation, and testing, saving both time and resources. Beyond efficiency, SDLC planning fostered coordination among stakeholders, faculty, students, and security staff by aligning their expectations with the system's capabilities. This structured approach emphasized that successful system development is not just about technical execution, but also about collaboration, foresight, and disciplined project management.